SECTION 4 | QUESTIONS WITH SOLUTIONS

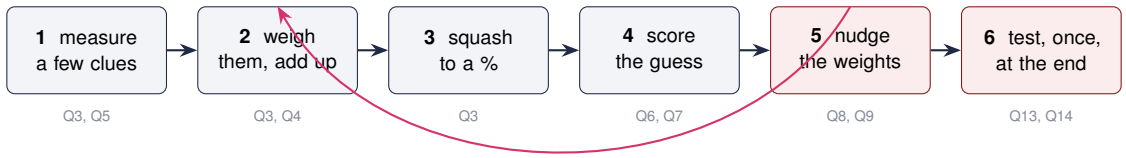
Teach Ollie

Logistic regression, end to end: features in, a probability out

THE BIG PICTURE, BEFORE ANY SYMBOLS

Forget the formulas for a second. Here is the *whole* idea, in six plain steps, using the sneaker photos as the example. Everything in this handout is really just one of these six steps, slowed down and looked at closely with real numbers.

> *Look at a photo. Measure a few simple things about it. Decide how much to trust each measurement, and add them up. Squeeze that total through a rule that always hands back a percentage – never anything else. Compare that percentage to the real answer, and give the guess a score. Nudge every weight a little, in whichever direction would have made the score better. Do that nudging thousands of times. Finally, and only once, check the result on photos the model has never seen before.*



Steps 2–5 repeat, again and again – Section 3 called this part "the climb".

That is genuinely the entire section. Steps 1–3 build the model (one forward pass, Question 3). Step 4 is just a way of keeping score (Questions 6–7). Steps 5–6 are how the model gets *better* at the score, one tiny nudge at a time (Questions 8–9). Everything else – picking a problem, designing your own clue, using every pixel, the final test – is this same six-step loop, applied a few different ways.

THE THREE PROBLEMS

Problem	Data	"Yes" means	Train / test
Is it a sneaker?	2,000 Fashion-MNIST photos	a sneaker	1,600 / 400
Is it a cat?	2,000 Quick, Draw! doodles (cat, bear, lion)	a cat	1,604 / 396
Will this lost dog get home?	2,000 Austin shelter records	its family reclaims it	1,568 / 432

Each problem has its own three features: **area, width, edges** for the photos and doodles; **named, fixed, age** for the dogs. The sneaker run is used for every worked number below; the cat and dog numbers sit alongside for comparison, exactly as the second, shorter run does in section.

NOTATION USED BELOW

You will not need to memorize any of this. These are just short names for the pieces in the six-step picture above – come back here any time a symbol below looks unfamiliar.

Symbol	Meaning
$x_0 = 1,\ x_1, x_2, x_3$	The always-on input and the three features of one example.
$\theta_0, \theta_1, \theta_2, \theta_3$	The weights: θ_0 is the **bias**, the rest belong to x_1, x_2, x_3. Together: θ.
$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$	The weighted sum. Can be any number, positive or negative.
$\sigma(z) = \dfrac{1}{1+e^{-z}}$	The **squash**. Turns any z into a number strictly between 0 and 1.
$p = \sigma(z)$	The model's output: the probability of "yes" for this example.
score (or LL)	$\log_{10}$ of the likelihood, averaged over the examples: $\frac{1}{n}\sum_i \left[y_i \log_{10} p_i + (1-y_i)\log_{10}(1-p_i)\right]$. Higher (closer to 0) is better.
slope	$\partial\,\mathrm{LL}/\partial\theta_j = \frac{1}{n}\sum_i (y_i - p_i)\, x_{ij}$: which way is uphill for that weight.
η	Learning rate (step size): 8 for sneakers, 3 for cats and dogs.

QUESTION 1 *warm-up*

Last section's recipe, in three words, was *score, slope, climb*. This week the model changes from "where am I?" to "is this a sneaker?". What stays the same, and what is new?

SOLUTION

In plain words: almost nothing changes. You still (1) score how good a guess is, (2) work out which way is uphill, and (3) take a small step that way. The only genuinely new thing this week is that the *answer* has to be a percentage – never a raw number – so one extra piece gets bolted on at the very end: a rule called the **squash** (σ), which turns any number at all into something between 0% and 100%.

Idea	Last section	This section
Parameters	$\theta = (\theta_1, \theta_2)$, a spot on the lawn	$\theta = (\theta_0, \theta_1, \theta_2, \theta_3)$, four weights
Score	$\log_{10}$P(pings caught)	$\log_{10}$P(real labels), same formula
Slope	tells which way is uphill	*same idea*: $(y - p) \times x$, averaged
Climb	step the spot uphill	step every weight uphill
New this week	—	σ: turns any sum into a probability

ANSWER The recipe (score, slope, climb) carries over unchanged. The new ingredient is σ, the squash that turns the weighted sum z into a probability p – that one piece is what makes this **logistic** regression.

QUESTION 2 *step 0: hold out a test set*

Ollie has 2,000 labeled photos and asks, "where should I start?" Before anything else, what should he do, and why does it matter?

SOLUTION

Step 1: see the problem

Think of it like an exam. If a student sees the exam questions *while* they are still studying, a perfect score proves nothing – you cannot tell if they actually understood the material, or just memorized the answer key. A model has exactly the same problem.

Step 2: fix it before doing anything else

Separate the data into a **training set** and a **test set**, chosen at random, *before* building or training anything. About a fifth is locked away and not looked at again until Question 14: 400 of 2,000 photos, 396 of 2,000 doodles, 432 of 2,000 dog records.



Step 3: follow one rule from here on

The test set is opened **once, at the end**, and never before. Peek at it early, even just to compare two models, and you start quietly "studying for the exam" without realizing it – the next real-world photo would not actually be new to your decisions anymore.

ANSWER Split the data first: training set for building and climbing, test set (about a fifth, chosen at random) locked away and checked only once, at the very end, to see how the model does on examples it has never seen.

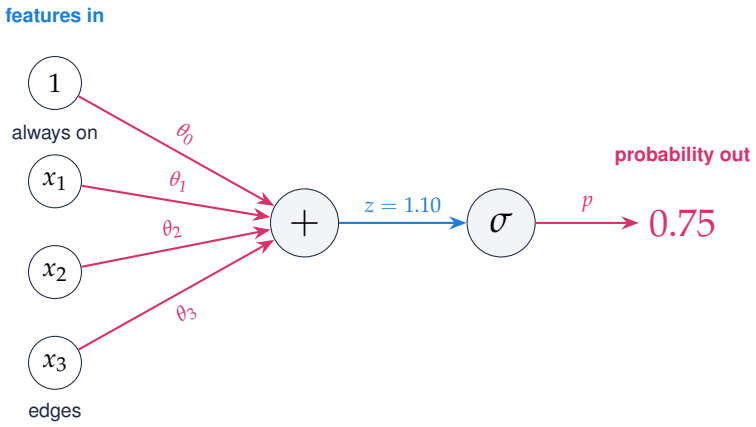
QUESTION 3 *step 1: the model*

Ollie wants a machine: a photo goes in, and the probability it is a sneaker comes out. Suppose, at some trial weights, the always-on input and the three toggled features give the weighted sum $z = 1.10$. What probability does the machine report, and what does the whole pipeline look like?

SOLUTION

This is steps 1–3 of the big picture, drawn out in full. In words, before any symbols: take the features, decide how much to trust each one, add them all up, then squeeze that total through σ so it comes out as a percentage. Always include one extra "feature" that is permanently switched on ($x_0 = 1$) – its weight, θ_0, is called the **bias**, and it is simply where the sum starts before any real feature has counted at all.

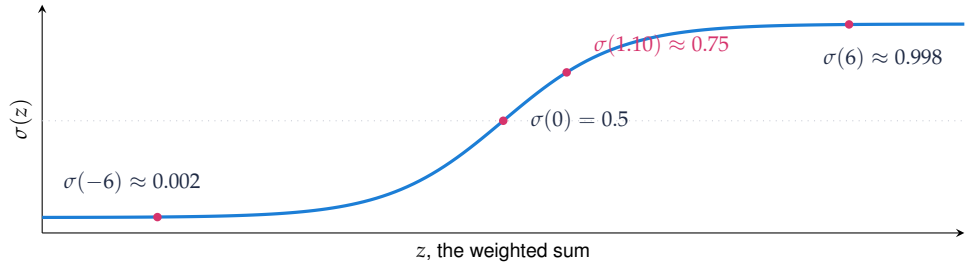
Step 1: weigh each clue and add them up



Step 2: squeeze that sum into a percentage

z can be any number at all – even negative. σ fixes that, by squashing it into something between 0 and 1:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma(1.10) = \frac{1}{1+e^{-1.10}} = \frac{1}{1+0.333} \approx 0.75$$



Step 3: read the result

No matter how large or how negative z gets, σ always lands strictly between 0 and 1 – that is exactly what makes the output a legal probability.

ANSWER $p = \sigma(1.10) \approx 0.75$: about a 3 in 4 chance this example is a "yes". The pipeline is always the same: features (with the always-on 1) in, weighted by θ, summed to z, squashed by σ, probability out.

QUESTION 4 *the five questions*

Ollie looks at the pipeline and asks five quick questions: (1) What are the weights? (2) What does a negative weight do? (3) Do the weights need to be between 0 and 1? (4) What is the always-on 1 for? (5) The sum can be any number – how is that a probability?

SOLUTION

Quick gut-check before moving on – all five answers follow from one idea: a weight is just "how much to trust a clue", and only the very last number in the pipeline (after σ) is forced to behave like a probability.

#	Answer	Why
1	**How much, which way**	How much each feature counts, and which way it pushes the probability.
2	**Pushes the chance down**	When that feature is on, it is a clue for "no".
3	**No**	A weight can be any number – big, small, or negative. Only the *output* p is a probability.
4	**Gives the bias** θ_0	$\theta_0 \times 1$ is where the sum starts before any feature counts at all.
5	σ **makes it one**	The sum is not a probability until it is squashed; σ turns any number into something strictly between 0 and 1.

ANSWER A weight's **sign** says which way it pushes (positive toward "yes", negative toward "no"); its **size** says how hard. Weights themselves are unrestricted – it is only the final squashed output that must be a legal probability.

QUESTION 5 *step 2: the features*

A feature is a number Ollie measures on each example. For the sneaker photos: *area*, *width*, *edges*. For the lost dogs: *named*, *fixed*, *age*. Why are features called "a choice", and what do the numbers look like for one sneaker and one dog?

SOLUTION

Step 1: know what a feature actually is

A feature is nothing fancy – just a number you decide to measure about each example. That is *all* the word means here.

Step 2: look at the real numbers

One sneaker vs. one non-sneaker, and one dog that got reclaimed vs. one that did not:

Sneaker photo	a sneaker	not
x_1 area	0.18	0.55
x_2 width	1.00	0.79
x_3 edges	0.16	0.16

Dog record	home	not
x_1 named	1	0
x_2 fixed	1	0
x_3 age/10	1.50	0.30

area = share of the frame covered; width = share of the frame's width spanned; edges = brightness jumps between neighboring pixels; age is the dog's years divided by 10.

Step 3: notice why this particular choice works

Sneakers sit **narrower on area** and **wider on width** than the rest of the catalog – that is the whole idea the model has to pick up from data. Features are a *choice*: area, width and edges were picked to be simple, not necessarily good. On the sneaker run they work well; on the cat run (below) the same recipe of features barely helps at all, because a cat, a bear and a lion doodle fill the frame about the same way.

ANSWER A feature is any number you choose to measure. A good feature is one where "yes" examples and "no" examples land on noticeably different numbers – area and width do that for sneakers; area, width and edges barely do it for cats.

QUESTION 6 *step 3: the likelihood*

Ollie's first guess at θ gives these five training photos the probabilities below. How do we turn these into one number that says whether θ is any good?

SOLUTION

Step 1: for every example, find the chance it gave the real answer

Label	p(sneaker)	chance of the real label	
sneaker	0.16	0.16	
not a sneaker	0.06	0.94	$= 1 - 0.06$
not a sneaker	0.17	0.83	$= 1 - 0.17$
sneaker	0.16	0.16	
not a sneaker	0.17	0.83	$= 1 - 0.17$

When the real label is "yes", we use p itself; when it is "no", we use $1 - p$.

Step 2: multiply them all together

$$\text{likelihood} = 0.16 \times 0.94 \times 0.83 \times 0.16 \times 0.83 \approx 0.017$$

That product is called the **likelihood** – how likely the whole training set is, under this θ. A good θ makes the real data likely, so *higher likelihood is better*.

> **Rounding note.** These five rows are a hand-sized snippet to show the mechanism. The full training set has 1,600 photos, and Ollie's first guess scores an *average* log-likelihood of -0.125 over all of them (Question 7) – not the same number as this five-row snippet, which exists only to make the arithmetic visible.

ANSWER Multiply the chance each example's *real* label got: p for a "yes", $1 - p$ for a "no". That product is the likelihood – the chance the whole training set would look the way it does, under this θ.

QUESTION 7 *taking logs*

With 1,600 training photos that is 1,600 chances multiplied. Won't that get impossibly tiny? What does Ollie's score actually look like?

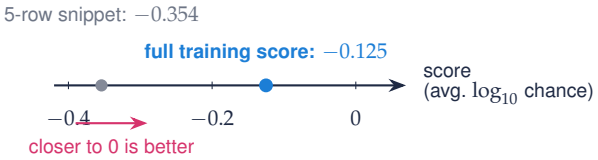
SOLUTION

Step 1: yes, it really would get that tiny

Multiplying hundreds of numbers that are each less than 1 shrinks the product to something far too small to read or compare sensibly.

Step 2: take the log, so multiplying becomes adding

$$\log_{10}(0.017) \approx -1.77, \qquad \text{score (average over the 5 rows)} = \frac{-1.77}{5} \approx -0.354$$



Step 3: read the number back as a chance

-0.125 is about a 75% typical per-example chance ($10^{-0.125} \approx 0.75$) – not a coincidence: it is close to the very first forward pass in Question 3. Scores stay negative, because every chance is below 1 – 0 would mean a perfect model. The course averages this log-likelihood over the training set (and, once, over the test set), in $\log_{10}$, exactly as Section 3's score was.

ANSWER Taking $\log_{10}$ turns an unreadable product into a sum we can actually track. The score is always negative; 0 is perfect, and *higher (closer to* 0*) is better*, same convention as the fog-game score in Section 3.

QUESTION 8 *step 4: find the slope*

Which way should Ollie move his four weights to raise the score? Take the first sneaker photo: the real answer is "yes" ($y = 1$), the machine currently says $p = 0.15$, and its features are $x_1 = 0.18$, $x_2 = 1$, $x_3 = 0.16$ (with $x_0 = 1$ always on).

SOLUTION

Step 1: compare the real answer to the guess

Look at one training example. Compare what *really* happened (y) to what the model currently guesses (p). If the model guessed too low on a "yes", push up every weight whose feature was switched on for that example; if it guessed too high, push those weights down. That is the whole idea.

Step 2: write that as a formula

The formula below is just a precise way of writing "push up if too low, down if too high, and push harder when the feature was strongly on":

$$\text{nudge for weight } j \;=\; (y_i - p_i)\, x_{ij}$$

Step 3: try it on one real photo

For this one photo, $y - p = 1 - 0.15 = 0.85$ (the model badly under-guessed), so every weight whose input is "on" gets nudged up, in proportion to that input:

Weight	input x_j	nudge $= (y - p)\, x_j$	direction
θ_0 (bias)	1	$0.85 \times 1 = +0.850$	up
θ_1 (area)	0.18	$0.85 \times 0.18 = +0.153$	up (small)
θ_2 (width)	1	$0.85 \times 1 = +0.850$	up
θ_3 (edges)	0.16	$0.85 \times 0.16 = +0.136$	up (small)

Step 4: do this for every example, then average

A pass through the whole training set averages one such nudge per example. A photo the machine already gets right (p close to y) barely moves anything, because $(y - p) \approx 0$ – the big nudges come from the photos the model is still getting *wrong*.

ANSWER Every weight moves by the average of $(y_i - p_i)\, x_{ij}$: up when the model under-predicts a "yes" (or over-predicts a "no"), scaled by how much that feature was switched on for that example.

QUESTION 9 *step 5: climb*

How does Ollie find the best θ, and how much does the score actually improve after 2,048 steps on each of the three problems?

SOLUTION

Step 1: apply the nudge

Take Question 8's nudge, average it over *every* training example, and actually apply it to the weights.

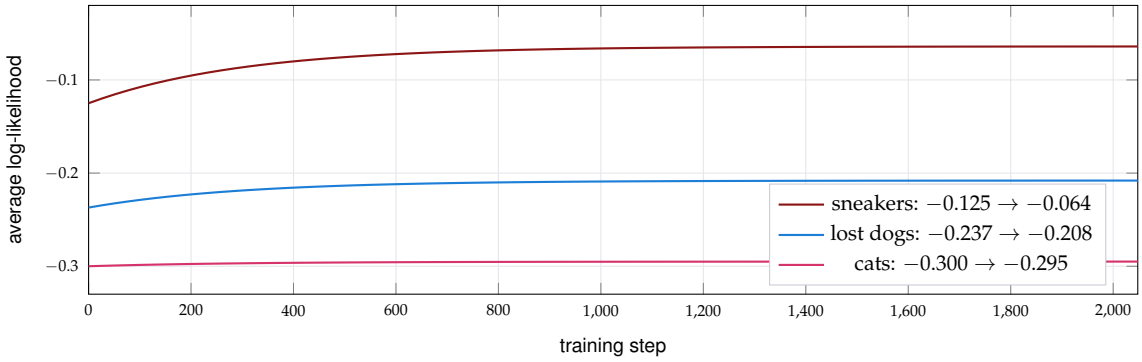
Step 2: repeat, thousands of times

Then do the whole thing again. And again. That repeated nudging is called **gradient ascent** – the exact same climbing idea as the fog game in Section 3, just climbing a different hill:

$$\theta_j \leftarrow \theta_j + \eta \times (\text{average nudge for } \theta_j)$$

with learning rate (step size) $\eta = 8$ for sneakers and $\eta = 3$ for cats and dogs, repeated 2,048 times.

Step 3: watch the score climb



Rounding note. The two checkpoints for each curve (step 0 and step 2,048) are the values the key reports; the smooth path between them is my own illustrative interpolation, not a published step-by-step log.

Step 4: compare the three problems

Sneakers climb the most, because area and width genuinely separate sneakers from the rest of the catalog. Cats barely move, because – as Question 5 already hinted – area, width and edges do not tell a cat doodle from a bear or lion doodle, so there is little uphill to find.

ANSWER Step every weight by $\eta\times$ its slope, over and over. Sneakers: $-0.125 \rightarrow -0.064$. Lost dogs: $-0.237 \rightarrow -0.208$. Cats: $-0.300 \rightarrow -0.295$ – almost flat, a first sign that the three chosen features are a poor fit for that problem.

QUESTION 10 *reading the trained weights*

After training, the three problems' weights came out very differently. Sneakers: area -13.14, width $+9.40$, edges -20.19. Cats: area -1.46, width $+1.22$, edges -3.22. Lost dogs: named $+3.29$, fixed $+0.83$, age $+1.17$. Why?

SOLUTION

After thousands of nudges (Question 9), the weights settle down. Reading them is simple once you know the rule: the **sign** tells you which way that clue pushes, and the **size** tells you how strongly.

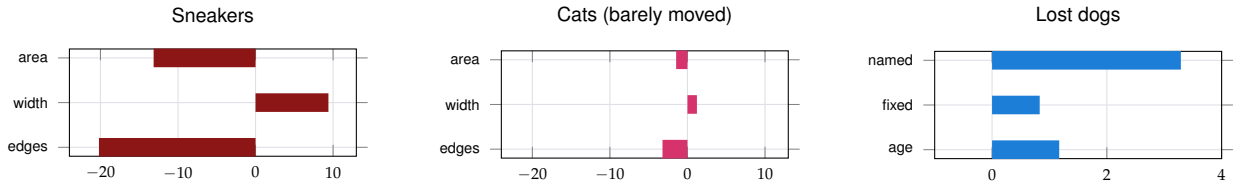
Step 1: read each weight in two parts – sign (which way) and size (how hard)

Sneakers	weight	$+0.1$ more	what it says
area	-13.14	-1.31	sneakers cover *less* of the frame than the rest of the catalog
width	$+9.40$	$+0.94$	sneakers are *wider*, relative to the frame, than most other items
edges	-20.19	-2.02	sneakers have *fewer* sharp edges than a boot or a bag

Cats	weight	$+0.1$ more	what it says
area	-1.46	-0.15	all three weights are small: a cat, a bear and a lion doodle cover the frame about the same way
width	$+1.22$	$+0.12$	widths are similar across the three too
edges	-3.22	-0.32	so no weight here can really tell them apart

Lost dogs	weight	switching on	what it says
named	$+3.29$	a 1: $+3.29$	96% of dogs that went home had a name – a tag or a chip is how a family gets found
fixed	$+0.83$	a 1: $+0.83$	a spayed/neutered dog is somewhat more likely to be reclaimed
age	$+1.17$	$+0.1$ more: $+0.12$	older dogs are somewhat more likely to be reclaimed

Step 2: see the same weights drawn as bars



Step 3: check it actually works

Plugging the real feature values from Question 5 into the trained sneaker weights (bias omitted, taken as 0, for a clean check):

$$z_{\text{sneaker}} = -13.14(0.18) + 9.40(1.00) - 20.19(0.16) \approx 3.80 \;\Rightarrow\; p \approx 0.978$$

$$z_{\text{not}} = -13.14(0.55) + 9.40(0.79) - 20.19(0.16) \approx -3.03 \;\Rightarrow\; p \approx 0.046$$

The trained model gives an actual sneaker a 98% chance and gives a non-sneaker only a 5% chance – exactly the separation training was climbing toward.

ANSWER A weight's **sign** says which way its feature pushes; its **size** says how hard, measured directly in "how much the score would rise if this feature nudged up by 0.1 (or switched from 0 to 1)". Small weights across the board (cats) mean the features themselves cannot tell the classes apart – no amount of training fixes a poor choice of features.

QUESTION 11 *design your own feature*

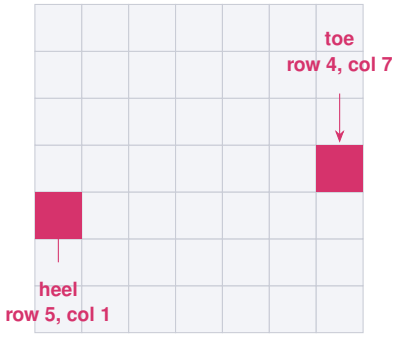
Ollie asks: "could I design a feature of my own? Which squares should I measure?" The room is shown the average sneaker and the average of everything else on a 7×7 grid, and clicks squares where the two differ. What makes a square worth picking, and how much did it help?

SOLUTION

Step 1: know what makes a square worth picking

Pick squares where the **average "yes"** picture and the **average "no"** picture look different; a square where they match carries no information.

Step 2: pick the squares



a sneaker touches these edges of the frame; almost nothing else does

Step 3: add it as a fourth feature, and train again

Ollie adds the picked squares' brightness as a fourth feature and trains again from where he was. On the **test set** (the fair judge – Question 13):

Problem	squares picked	right, before → after	score, before → after
Sneakers	toe + heel (above)	369 → 378 of 400	−0.075 → −0.060
Cats	the gap between the ears	236 → 307 of 396	−0.291 → −0.224
Lost dogs	"healthy" (a record column)	318 → 318 of 432	−0.228 → −0.227

Step 4: judge it honestly

The cat gap-between-the-ears square is dark for cats and bright for a bear or lion's rounder head, so it gets a large negative weight and very nearly catches up to every pixel (307 vs. every pixel's 319, below). The dogs' "healthy" column barely moves the needle: whether a feature helped is judged on the *test* set, not on how clever it sounds.

ANSWER A good hand-picked feature is one where the two classes' averages visibly differ. It can help a lot (cats: close to every pixel, from three weak features) or almost nothing (dogs' "healthy"), and the only honest way to tell is the held-out test score.

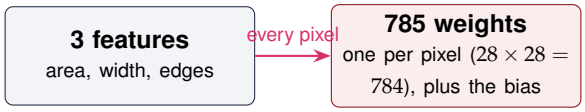
QUESTION 12 *every pixel*

"What if every pixel were a feature? How many weights would my machine need?"

SOLUTION

Step 1: stop hand-picking, use everything

Instead of hand-picking one or two clues like Question 11, why not just use *every* clue available? For a 28×28 photo, that means every single pixel becomes its own feature:



Step 2: train it exactly the same way as before

Training all 785 weights at once works exactly the same way – just more of it (3,000 steps instead of 2,048). On sneakers:

Model	training score	test: right of 400
3 features	−0.064	369
every pixel (785 weights)	−0.026	390

Step 3: check it is not just memorizing

More features can help *or* just memorize the training set – the only way to tell the difference is to check on data the model has never seen. Here every pixel does better on the *test* set too, so 785 weights are finding real shape, not just memorizing. The dog records have no pixels, so that run skips this step.

ANSWER 785: a weight for each of the 784 pixels, plus the bias. Every pixel wins on sneakers and cats because it can trace shape in a way three hand-measured numbers cannot – but "more features" is not automatically better; it only counts if the *test* score rises too.

QUESTION 13 *the test set rule*

"Can I check my models on the test set now?" Ollie has just trained several models and wants to know which is best. When should he look?

SOLUTION

Step 1: the rule

Once. At the end. Not after every change.

Step 2: why

If you peek at the test set, compare two models, and keep the one that scores better, you are no longer choosing blind – you are *fitting to the test set itself*, the same mistake as training on it. The next "new" photo it sees would not really be new to your decisions anymore. All model-picking (which features, how long to train, which squares to click) happens using only the **training** score; the test set is opened exactly once, after every decision is already made, purely to report how things turned out.

ANSWER Once, at the end, after every choice about features and training is already locked in – never used to pick between models along the way.

QUESTION 14 *the big reveal: the test table*

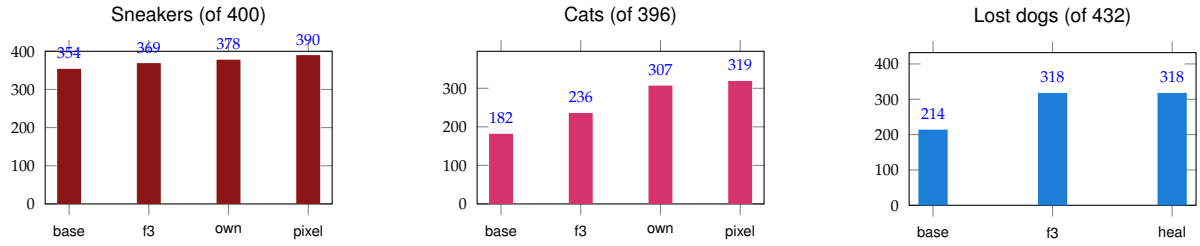
Ollie finally opens the test set. What does each model get right, and what does the room notice?

SOLUTION

Step 1: open the locked-away exam

Now, and only now (Question 13), do we look at it. Every model below was trained without ever seeing these photos:

Step 2: compare every model, side by side



base = baseline (always guesses the more common label); 3f = the 3 hand-measured features; +own/+heal = with the room's own feature added; pixel = every pixel. Exact log-likelihoods are in the key; counts are shown here.

Step 3: notice the pattern

Every bar is taller than its baseline, and the bars generally grow from left to right – but "grow" is not guaranteed, it is *earned* on this chart, example by example.

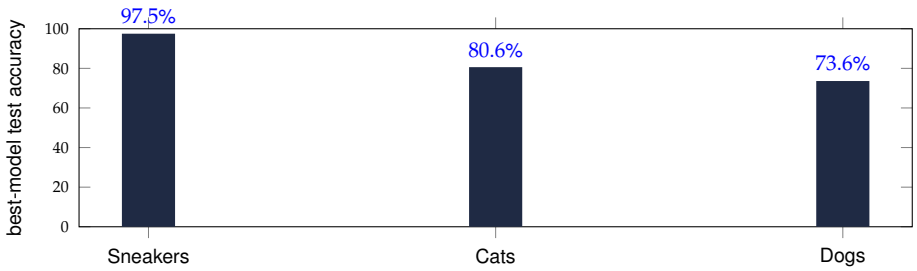
ANSWER Every model beats its baseline. Every pixel wins on new photos too (not just training), so it is not merely memorizing. A hand-picked feature counts as a real help only when the *test* score rises – it did for sneakers and especially cats, barely for the dogs' "healthy" column. "More features always overfit" is not true here.

QUESTION 15 *second run: three bets*

On a second, shorter problem Ollie does everything the room already taught him, and instead asks three bets: which feature differs most, which way its weight will go, and which problem was hardest for the machine.

SOLUTION

Bet	What the numbers show
Which feature differs most?	Sneakers: area, 0.27 (sneakers) vs. 0.45 (rest). Dogs: named, 0.96 vs. 0.45. Cats: *none* – they look about the same, and that is the right tile.
Which way will the weight go?	Sneakers: negative for area (sneakers cover less of the frame). Dogs: positive for named. Cats: skipped – no feature differs, so there is nothing to predict.
Which problem was hardest?	Compare best-model test accuracy: sneakers $390/400 \approx 98\%$, cats $319/396 \approx 81\%$, dogs $318/432 \approx 74\%$. Lowest share is hardest.



ANSWER Lost dogs are the hardest problem: a shelter record with three or four columns says far less about a dog than 784 pixels say about a shoe, so even the best dog model tops out around 74%.

QUESTION 16 *exit ticket*

One word each. (1) A weight of -20: push the probability up or down? (2) σ turns any sum into a number between what and what? (3) Checking the test set after every tweak: good idea or bad?

SOLUTION

	Answer	Why
1	**Down**	Negative weights are a clue for "no".
2	**0 and 1**	That is exactly what makes $\sigma(z)$ a legal probability.
3	**Bad**	You would be tuning to the exam. Check once, at the end.

ANSWER Down; between 0 and 1; bad. The week in one line: features in, weighted and summed, squashed to a probability, scored by likelihood, climbed by gradient ascent, and judged – once – on data it has never seen.

COMMON QUESTIONS AND MISTAKES TO WATCH FOR

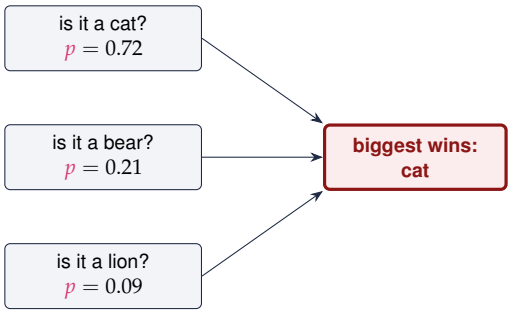
Question or mistake	Answer
Why is the log-likelihood negative?	It is the log of a chance, and a chance is below 1. Closer to 0 is better; 0 would be perfect.
Is a weight a probability?	No. A weight can be any number, big, small, or negative. Only the *output*, after σ, is a probability.
Why not check the test set after every change?	Then you would be tuning to it, and it would stop being new. Once, at the end.
Why do the three features barely help on cats?	Cats, bears and lions all fill the frame about the same. The averages match, so no weight can tell them apart – a poor choice of feature, not too little training.
Why does every pixel win?	785 weights can trace shape that 3 numbers cannot. It also does well on *new* photos, so it is not just memorizing here.
Where is θ_0?	The always-on 1, times θ_0: where the weighted sum starts before any feature counts at all.
Can "right" and the log-likelihood disagree?	Yes – the dogs' "healthy" feature left the count of right calls unchanged but still nudged the log-likelihood up a little: probabilities moved, but none crossed 0.5.
Does more features always mean overfitting?	No. Here every extra feature was judged on the *test* set, and every one of them helped or did no harm – "more features always overfit" is not a rule.

QUESTION 17 *if you have extra time*

(a) What if there were three kinds – cat, bear, lion – instead of yes/no? Ollie's machine only ever says yes or no. (b) Doodles are 28×28. What about a real photo, $4{,}000 \times 3{,}000$?

SOLUTION

(a) Build **one machine per class**, each answering its own yes-or-no question ("is it a cat?", "is it a bear?", "is it a lion?"), and let the **biggest probability win**.



(b) A $4{,}000 \times 3{,}000$ photo has $4{,}000 \times 3{,}000 = 12{,}000{,}000$ pixels – twelve million features, and so twelve million and one weights. The mechanics (weighted sum, squash, climb) stay exactly the same; only the size of θ changes.

ANSWER (a) One yes/no machine per class, then take the largest probability. (b) 12 million pixel-features (plus the bias) – same recipe, much bigger θ.